

Backend Integration Guide – Step by Step (Beginner-Friendly)

This guide explains how to add a backend to your InvoiceBill React Native app, based on the Product Requirements Document (PRD).

Table of Contents

1. [What You Need to Understand First](#1-what-you-need-to-understand-first)
2. [Backend Architecture Overview](#2-backend-architecture-overview)
3. [Step 1: Download and Install Prerequisites](#3-step-1-download-and-install-prerequisites)
4. [Step 2: Choose Your Database](#4-step-2-choose-your-database)
5. [Step 3: Create the Backend Project](#5-step-3-create-the-backend-project)
6. [Step 4: Set Up Database and Core Services](#6-step-4-set-up-database-and-core-services)
7. [Step 5: APIs You Need to Build](#5-step-5-apis-you-need-to-build)
8. [Step 6: Connect React Native App to Backend](#6-step-6-connect-react-native-app-to-backend)
9. [Step 7: External Integrations (Neokred, Mobiquick, etc.)](#7-step-7-external-integrations)
10. [Security Checklist](#8-security-checklist)
11. [Deployment Options](#9-deployment-options)

1. What You Need to Understand First

What is a Backend?

Your React Native app (the **frontend**) runs on the user's phone. The **backend** is a server that:

- Stores data (users, invoices, transactions)
- Processes payments
- Talks to external APIs (banks, SMS, etc.)
- Sends push notifications

Your app sends requests to the backend (e.g., "create invoice", "send payment"), and the backend responds.

How They Talk

...

[Your App] ---HTTP/HTTPS---> [Backend Server] ---> [Database]

|

+---> [External APIs: Neokred, SMS, etc.]

...

2. Backend Architecture Overview (from PRD)

Recommended Stack:

- **Runtime:** Node.js (JavaScript on the server)
- **Framework:** NestJS (structured, similar to Angular)
- **Database:** PostgreSQL (stores users, invoices, transactions)
- **Cache:** Redis (fast temporary storage, sessions)
- **Hosting:** AWS (or alternatives like Railway, Render, DigitalOcean)

Layers:

...

Mobile App (React Native)

↓

API Gateway (handles requests)

↓

Services (Auth, Payment, Invoice, Transaction, User)

↓

Database + Redis + External APIs

...

3. Step 1: Download and Install Prerequisites

3.1 Node.js

- **What:** Runs JavaScript on your computer and server
- **Where to download:** <https://nodejs.org/>
- **Choose:** LTS (Long Term Support) version
- **After install:** Open terminal and run `node -v` and `npm -v` to confirm

3.2 PostgreSQL (Database)

- **What:** Stores all your app data
- **Where to download:** <https://www.postgresql.org/download/>
- **Or use cloud:** [Supabase](<https://supabase.com>) (free tier) or [Neon](<https://neon.tech>) (PostgreSQL in cloud – easier for beginners)

3.3 Redis (Optional for start)

- **What:** Cache layer, sessions
- **Where to download:** <https://redis.io/download/>
- **For beginners:** Skip initially; add later when scaling

3.4 Git

- **What:** Version control
- **Where to download:** <https://git-scm.com/downloads>

4. Step 2: Choose Your Database

| Option | Best For | Setup Difficulty |

Supabase	Beginners, free tier	Very easy	
Neon	Cloud PostgreSQL, free tier	Easy	
Local PostgreSQL	Full control	Medium	
Railway / Render	Host DB + backend together	Easy	

Recommendation for beginners: Start with **Supabase** – it gives you PostgreSQL + Auth + Storage in one.

5. Step 3: Create the Backend Project

5.1 Create a NestJS Project

```
```bash
```

```
Install NestJS CLI globally
```

```
npm i -g @nestjs/cli
```

```
Create new backend project (run this in a NEW folder, not inside InvoiceBill)
```

```
nest new invoicebill-backend
```

```
When prompted, choose npm
```

```
cd invoicebill-backend
```

```
```
```

5.2 Project Structure (What You Get)

```
```
```

```
invoicebill-backend/
```

```

└─ src/
 └─ app.module.ts
 └─ main.ts # Entry point
 └─ ...
└─ package.json
└─ ...
...

```

## ## 6. Step 4: Set Up Database and Core Services

### ### 6.1 Install Database Driver

```

```bash
npm install @nestjs/typeorm typeorm pg
...

```

6.2 Create Database Tables (Schema)

You need tables for:

- **users** (id, email, phone, password_hash, created_at)
- **bank_accounts** (id, user_id, account_number, ifsc, bank_name, is_default)
- **invoices** (id, user_id, client_name, amount, items, status, pdf_url)
- **transactions** (id, user_id, type, amount, status, reference)
- **customers** (id, user_id, name, email, phone)
- **items** (id, user_id, name, rate, description)

6.3 Connect to PostgreSQL

In `app.module.ts`:`

```
``typescript
```

```
import { TypeOrmModule } from '@nestjs/typeorm';
```

```
@Module({
```

```
  imports: [
```

```
    TypeOrmModule.forRoot({
```

```
      type: 'postgres',
```

```
      host: process.env.DB_HOST || 'localhost',
```

```
      port: 5432,
```

```
      username: process.env.DB_USER || 'postgres',
```

```
      password: process.env.DB_PASSWORD,
```

```
      database: process.env.DB_NAME || 'invoicebill',
```

```
      entities: [__dirname + '**/*.entity{.ts,.js}'],
```

```
      synchronize: true, // For dev only; use migrations in production
```

```
    }),
```

```
  ],
```

```
})
```

```
export class AppModule {}
```

```
...
```

```
---
```

7. Step 5: APIs You Need to Build

Based on the PRD and your app, you need these API endpoints:

7.1 Authentication

Method	Endpoint	Purpose
--------	----------	---------

-----	-----	-----
-------	-------	-------

POST	/auth/register	Sign up new user
------	----------------	------------------

	POST		`/auth/login`		Login (returns JWT token)	
	POST		`/auth/otp/send`		Send OTP to phone/email	
	POST		`/auth/otp/verify`		Verify OTP	
	POST		`/auth/refresh`		Refresh JWT token	

7.2 Users

	Method		Endpoint		Purpose	
	-----		-----		-----	
	GET		`/users/me`		Get current user profile	
	PATCH		`/users/me`		Update profile	

7.3 Bank Accounts

	Method		Endpoint		Purpose	
	-----		-----		-----	
	GET		`/bank-accounts`		List user's bank accounts	
	POST		`/bank-accounts`		Add bank account	
	PATCH		`/bank-accounts/:id`		Update / set default	
	DELETE		`/bank-accounts/:id`		Remove account	

7.4 Invoices

	Method		Endpoint		Purpose	
	-----		-----		-----	
	GET		`/invoices`		List user's invoices	
	POST		`/invoices`		Create invoice	
	GET		`/invoices/:id`		Get single invoice	
	PATCH		`/invoices/:id`		Update invoice	
	GET		`/invoices/:id/pdf`		Download PDF	
	POST		`/invoices/:id/share`		Generate share link	

7.5 Transactions

	Method		Endpoint		Purpose	
--	--------	--	----------	--	---------	--

Method	Endpoint	Purpose
GET	/transactions	List transactions (with filters)
GET	/transactions/:id	Get transaction details

7.6 Payments

Method	Endpoint	Purpose
POST	/payments/send	Send money
POST	/payments/qr	Initiate QR payment
GET	/payments/status/:id	Check payment status

7.7 Customers / Contacts

Method	Endpoint	Purpose
GET	/customers	List customers
POST	/customers	Add customer
PATCH	/customers/:id	Update customer
DELETE	/customers/:id	Delete customer

7.8 Items (for invoices)

Method	Endpoint	Purpose
GET	/items	List items
POST	/items	Add item
PATCH	/items/:id	Update item
DELETE	/items/:id	Delete item

8. Step 6: Connect React Native App to Backend

8.1 Install Axios (HTTP client)

```
```bash
cd m:/REACT\ NATIVE/InvoiceBill
npm install axios
```
```

8.2 Create API Service

Create `src/services/api.ts`:

```
```typescript
import axios from 'axios';

const API_BASE = 'http://YOUR_BACKEND_URL:3000'; // Replace with your server URL

export const api = axios.create({
 baseURL: API_BASE,
 headers: { 'Content-Type': 'application/json' },
});

// Add auth token to every request
api.interceptors.request.use((config) => {
 const token = /* get from AsyncStorage */;
 if (token) config.headers.Authorization = `Bearer ${token}`;
 return config;
});
```
```

8.3 Example: Fetch Invoices

```
``typescript
```

```
const response = await api.get('/invoices');
```

```
setInvoices(response.data);
```

```
``
```

8.4 For Local Development

- Run backend: `npm run start:dev` (in backend folder)
- Backend runs at `http://localhost:3000`
- For Android emulator, use `http://10.0.2.2:3000`
- For iOS simulator, use `http://localhost:3000`
- For physical device, use your computer's LAN IP (e.g. `http://192.168.1.5:3000`)

9. Step 7: External Integrations (APIs You Need)

9.1 Neokred

- **What:** Banking and payment APIs
- **Where:** <https://neokred.com/> – sign up for API access
- **You get:** APIs for bank verification, payments, account linking
- **Use for:** Bank account verification, payouts, UPI

9.2 Mobiquick

- **What:** Wallet and payment services
- **Where:** Contact Mobiquick for partnership
- **Use for:** Wallet balance, load money, transfers

9.3 PanicDrop

- **What:** Payout/logistics APIs
- **Where:** Contact for integration
- **Use for:** Agent payouts, logistics

9.4 Firebase

- **What:** Push notifications, analytics
- **Where:** <https://console.firebase.google.com/>
- **Download:** `npm install @react-native-firebase/app @react-native-firebase/messaging`
- **Use for:** Push notifications when payment received, invoice paid

9.5 SMS Gateway (for OTP)

| Provider | Link | Notes |

|-----|-----|-----|

| MSG91 | <https://msg91.com> | Popular in India |

| Twilio | <https://twilio.com> | Global |

| AWS SNS | Part of AWS | If using AWS |

10. Security Checklist

1. **HTTPS:** Always use HTTPS in production
2. **JWT:** Store tokens securely (e.g. `react-native-keychain`)
3. **Environment variables:** Never commit API keys – use `.env`
4. **Validation:** Validate all inputs on backend
5. **Rate limiting:** Prevent abuse (e.g. `@nestjs/throttler`)
6. **Encryption:** Encrypt sensitive data (AES for fields like account numbers)

11. Deployment Options

| Platform | Difficulty | Free Tier | Best For |

|-----|-----|-----|-----|

| **Railway** | Easy | Yes | Quick start |
| **Render** | Easy | Yes | Simple hosting |
| **AWS** | Hard | 12 months free | Production, scale |
| **DigitalOcean** | Medium | No | Predictable pricing |
| **Vercel** | Easy | Yes | API routes (Node) |

Beginner path: Deploy backend on **Railway** or **Render**, database on **Supabase**.

12. Recommended Order of Implementation

1. **Week 1:** Set up NestJS project + PostgreSQL (Supabase)
2. **Week 2:** Auth APIs (register, login, OTP) + User APIs
3. **Week 3:** Bank Accounts, Customers, Items APIs
4. **Week 4:** Invoices CRUD + PDF generation
5. **Week 5:** Transactions APIs, connect app to backend
6. **Week 6:** Payments (mock first, then Neokred)
7. **Week 7:** Firebase notifications
8. **Week 8:** Deploy to Railway/Render

13. Quick Links Summary

| What | Where |
|-----|-----|
| Node.js | <https://nodejs.org> |
| NestJS Docs | <https://docs.nestjs.com> |
| PostgreSQL | <https://postgresql.org> or Supabase |
| Supabase | <https://supabase.com> |

| Neokred | <https://neokred.com> |
| Firebase | <https://firebase.google.com> |
| Railway | <https://railway.app> |
| Render | <https://render.com> |

14. Next Steps

1. Install Node.js and create a NestJS backend project
2. Sign up for Supabase and get your database URL
3. Implement Auth APIs first (register, login)
4. Connect your React Native app using Axios
5. Replace mock data in the app with real API calls

If you want, I can help you with specific steps (e.g. Auth API code, connecting invoices to the backend, or Supabase setup).